



CHECKPOINT 6 · ENTREGA FINAL

ENTREGA FINAL

FIAP · Mobile Development & IoT · Engenharia de Software

O que mudou do protótipo para o produto, as decisões técnicas finais e como o APK é gerado.

Disciplina: Mobile Development & IoT · FIAP · 3º ano de Engenharia de Software

Etapa: Checkpoint 6 — Entrega final

1. O QUE MUDOU DO CP5 PARA O CP6

O CP5 entregou um protótipo navegável com dados mockados e banco ligado. O CP6 fechou as três lacunas que o próprio documento do protótipo tinha deixado registradas.

ÁREA	NO CP5	NO CP6
Catálogo de filmes	Nove filmes num JSON dentro do aplicativo	API pública do TMDB: o acervo inteiro, em português, com pôsteres e provedores de streaming do Brasil
Onde o filme vive	Arquivo no repositório	Tabela <code>movies</code> no Postgres, alimentada pela escolha do curador
Banco fora do ar	Tela vazia, sem explicação	Queda para os dados locais, com aviso na tela
Distribuição	Rodava em <code>npm run web</code> ou Expo Go	APK instalável, gerado pelo EAS Build

1.1 A busca de filmes

A mudança mais visível está na tela do curador. Antes, escolher o filme era escolher entre nove. Agora a busca consulta o TMDB e devolve qualquer filme já lançado, com o pôster e os gêneros corretos.

A busca foi desenhada para ser barata: ela espera a digitação parar por 400ms antes de sair para a rede, e traz só o necessário para a lista. Duração e plataforma de streaming — que exigem duas chamadas a mais — são buscadas apenas para o filme que o curador realmente escolhe.

Isso atende o **desafio bônus** proposto no enunciado: integração com uma API pública real.

1.2 O catálogo virou tabela

Com filmes vindos de fora, guardar só o identificador na rodada não bastava: na semana seguinte o aplicativo não saberia dizer o nome do que o clube assistiu. A migração

`20261001165252_catalogo_de_filmes.sql` cria a tabela `movies`, e escolher um filme passa a ser também guardá-lo.

Os nove filmes herdados do CP4 entram na tabela com `poster_url` nulo de propósito: a arte deles está no repositório desde o CP4 e continua sendo usada, o que mantém as telas idênticas às conceituais.

1.3 O aplicativo não depende mais do banco para abrir

O Supabase gratuito **suspende projetos parados por uma semana** — e foi exatamente o que aconteceu com este entre a entrega do CP5 e a do CP6. Um APK na mão de outra pessoa pode ser aberto muito depois da última vez que alguém mexeu no banco.

Antes, nessa situação, o aplicativo abria em branco. Agora ele desce para o catálogo local, continua inteiramente navegável e diz o que aconteceu. A decisão vale para qualquer falha de rede, não só para a pausa.

2. DECISÕES TÉCNICAS FINAIS

DECISÃO	ALTERNATIVA DESCARTADA	POR QUÊ
Busca no TMDB, catálogo no Postgres	Buscar no TMDB toda vez que uma tela precisa de um filme	A rodada precisa saber o nome do filme mesmo offline, e repetir a chamada a cada tela gastaria cota da API sem necessidade
Chave do TMDB opcional	Exigir a chave para o aplicativo subir	Quem clona o repositório sem chave ainda consegue rodar tudo. Uma configuração ausente não deve ser motivo de aplicativo quebrado
Queda para dados locais	Mostrar uma tela de erro com "tentar de novo"	Um clube de cinema não é um app bancário: mostrar o conteúdo que se tem é mais útil do que bloquear a tela. O aviso mantém a honestidade
APK em vez de App Bundle	<code>app-bundle</code> , o padrão da Play Store	O enunciado pede um arquivo instalável. App Bundle só é instalável passando pela loja
<code>appVersionSource: remote</code>	Versionar à mão no <code>app.json</code>	O EAS incrementa o <code>versionCode</code> sozinho a cada build, o que evita conflito ao reenviar
Sem autenticação	Login por e-mail no Supabase Auth	Fora do escopo das três entregas. As políticas de RLS já estão no lugar, permissivas, prontas para passar a verificar o usuário quando houver login

3. COMO O APK É GERADO

O build roda na nuvem do Expo, a partir de `eas.json`.

```
npx eas-cli login
npx eas-cli build --platform android --profile preview
```

O perfil `preview` produz um **APK de distribuição interna** — instalável direto, sem passar pela Play Store. O `production` também gera APK, pelo mesmo motivo do enunciado.

3.1 As variáveis de ambiente

O `.env` não vai para o repositório, e por isso também não sobe para o servidor de build. As duas chaves precisam existir no ambiente do EAS:

```
npx eas-cli env:create --environment production --name EXPO_PUBLIC_SUPABASE_URL --value "https://  
npx eas-cli env:create --environment production --name EXPO_PUBLIC_SUPABASE_ANON_KEY --value "sua-  
npx eas-cli env:create --environment production --name EXPO_PUBLIC_TMDB_API_KEY --value "sua-cha
```

As três são chaves de cliente: o Supabase protege o acesso pelas políticas de RLS, não por esconder a chave anônima. A `service_role` nunca entra aqui.

4. O QUE FICOU DE FORA, E POR QUÊ

Registrar isto é parte da entrega: um produto honesto diz onde termina.

- **Criar clube e entrar por código** — a tela de abertura tem os dois botões, mas eles entram no clube de demonstração. O fluxo real depende de autenticação, que está fora do escopo das três entregas.
- **Os outros membros são simulados** — confirmam presença e votam com notas fixas. Com login, virariam pessoas de verdade; o código que simula está isolado em `src/data/simulation.ts` e marcado como tal.
- **Sem atualização em tempo real** — duas telas abertas compartilham o mesmo banco, mas a segunda só vê a mudança ao recarregar. O Supabase oferece Realtime; ligá-lo é pequeno, e foi deixado de fora para não mexer em comportamento na véspera da entrega.
- **Notificações** — "sua vez de escolher" e "sessão hoje" estavam previstas no escopo como *importantes*, não essenciais. O produto funciona sem elas; com elas, funcionaria melhor.